

The MUMPS Primer: An Introduction and Tutorial for the 1976 Standard

Table of Contents

1. Introduction	4
1.1. What is MUMPS?	4
1.2. The 1976 Standard	5
1.3. Who This Book Is For	6
1.4. Getting Started	6
2. The Language at a Glance	6
2.1. Everything Is a String	7
2.2. No Operator Precedence	7
2.3. Line-Oriented Structure	7
2.4. Commands Are Terse	7
2.5. The Persistent Database Is Built In	7
3. Variables and Data Types	8
3.1. Local Variables	8
3.2. Subscripted Variables	8
3.3. Strings, Numbers, and Automatic Conversion	8
3.4. Truth Values	9
4. Expressions and Operators	9
4.1. Strict Left-to-Right Evaluation	9
4.2. Arithmetic Operators	9
4.3. Unary Operators	10
4.4. Relational Operators	10
4.5. Logical Operators	11
4.6. The Concatenation Operator	11
4.7. Pattern Match	11
5. Commands	12
5.1. SET — Assign a Value	12
5.2. WRITE — Output Data	12
5.3. READ — Input Data	13
5.4. IF and ELSE — Conditional Execution	14
5.5. Post-Conditionals	14
5.6. FOR — Repetition	15
5.7. DO — Call a Subroutine	16

5.8. GOTO — Transfer Control	17
5.9. QUIT — Return from Subroutine or Exit FOR	17
5.10. HALT and HANG	18
5.11. XECUTE — Execute a String as Code	18
5.12. KILL — Delete Variables	18
5.13. BREAK	19
6. String Functions	19
6.1. \$LENGTH — String Length	19
6.2. \$EXTRACT — Substring by Position	19
6.3. \$PIECE — Substring by Delimiter	20
6.4. \$FIND — Search for Substring	20
6.5. \$JUSTIFY — Right-Justify and Format Numbers	21
6.6. \$ASCII and \$CHAR — Character Code Conversion	21
6.7. \$RANDOM — Random Number	21
6.8. \$SELECT — Conditional Expression	22
6.9. \$TEXT — Retrieve Source Line	22
7. The Global Database	22
7.1. Global Variables	22
7.2. Building a Data Model	23
7.3. \$DATA — Testing Whether a Node Exists	23
7.4. \$NEXT — Walking the Tree	24
7.5. The Naked Reference	25
7.6. KILL on Globals	25
8. Input/Output	25
8.1. Formatting Output	25
8.2. The Special Variables \$X and \$Y	26
8.3. Device I/O (Overview)	26
9. Indirection	27
9.1. Name Indirection	27
9.2. Subscript Indirection	27
9.3. Argument Indirection	27
9.4. Pattern Indirection	27
10. Special Variables	28
10.1. \$HOROLOG — Date and Time	28
10.2. \$IO — Current Device	28
10.3. \$JOB — Process Identifier	28
10.4. \$STORAGE — Available Space	28
10.5. \$TEST — Last Test Result	29
10.6. \$X and \$Y — Cursor Position	29
11. Routines and Program Structure	29
11.1. Routine Structure	29

11.2. Lines and Labels	29
11.3. Entry References	30
11.4. Command Abbreviations	30
12. Putting It All Together	31
13. Command Reference	32
13.1. BREAK	32
13.2. CLOSE	32
13.3. DO	33
13.4. ELSE	33
13.5. FOR	33
13.6. GOTO	34
13.7. HALT	34
13.8. HANG	34
13.9. IF	35
13.10. KILL	35
13.11. LOCK	36
13.12. OPEN	36
13.13. QUIT	36
13.14. READ	37
13.15. SET	37
13.16. USE	38
13.17. VIEW	38
13.18. WRITE	38
13.19. XECUTE	39
14. Function Reference	39
14.1. \$ASCII	39
14.2. \$CHAR	40
14.3. \$DATA	40
14.4. \$EXTRACT	40
14.5. \$FIND	41
14.6. \$JUSTIFY	41
14.7. \$LENGTH	42
14.8. \$NEXT	42
14.9. \$PIECE	42
14.10. \$RANDOM	43
14.11. \$SELECT	43
14.12. \$TEXT	43
15. Special Variables Reference	44
15.1. \$HOROLOG	44
15.2. \$IO	44
15.3. \$JOB	44

15.4. \$STORAGE	44
15.5. \$TEST	45
15.6. \$X	45
15.7. \$Y	45
16. Operators Reference	45
16.1. Unary Operators	45
16.2. Binary Operators	45
17. Syntax Summary	47
17.1. Line Structure	47
17.2. Routine Structure	47
17.3. Name Syntax	47
17.4. Variable Syntax	47
17.5. Numeric Literal Syntax	48
17.6. String Literal Syntax	48
17.7. Entry Reference Syntax	48
Appendix A: ASCII Code Table	48
Appendix B: Glossary	49
18. What We Have Not Covered	51
References	51

1. Introduction

1.1. What is MUMPS?

In 1966, at the Laboratory of Computer Science at Massachusetts General Hospital in Boston, Octo Barnett, Neil Pappalardo, and Curtis Marble created a programming language called MUMPS — the *Massachusetts General Hospital Utility Multi-Programming System*.

MUMPS was born from a practical need: hospitals required a system where doctors and nurses could simultaneously access and update patient data from multiple terminals, in real time, on affordable hardware. In 1966, "affordable hardware" meant a DEC PDP-7 with 8K words of memory and a small disk. There was no room for separate layers — no separate operating system, file system, database server, and application language stacked on top of each other. Everything had to be one integrated system.

What emerged was something remarkable: a programming language with a built-in hierarchical database — what the database world would not have a name for until four decades later: a *NoSQL* database with an integrated programming language.

The core data abstraction in MUMPS is the *global variable*: a hierarchical, sparse array that is stored persistently on disk and shared among all concurrent users. In today's terms, globals are a schema-free, ordered key-value store with composite keys, created on first assignment and deleted on demand.

```
SET ^PATIENT(12345,"NAME")="Smith, John"  
SET ^PATIENT(12345,"DOB")="1945-03-15"  
SET ^PATIENT(12345,"LAB","2024-01-15","GLUCOSE")=95
```

There is no **CREATE TABLE**, no schema definition, no **INSERT** statement. You simply SET a value at a hierarchical path, and the system creates whatever intermediate nodes are necessary. KILL a node, and the entire subtree beneath it disappears.

Consider what MUMPS had in 1966 that the rest of the computing world would spend decades reinventing:

- **Schema-free storage:** No declarations required. Nodes are created on first assignment. This is what MongoDB would call "schemaless documents" in 2009.
- **Hierarchical data model:** Patient records naturally form trees. MUMPS globals mirror this structure directly. This is what Firebase would call "real-time hierarchical data" in 2012.
- **Ordered key traversal:** The **\$NEXT** function walks through sibling nodes in sorted order without an explicit index — what Redis would call "sorted sets" in 2009, or LevelDB "ordered iteration" in 2011.
- **Integrated language and database:** There is no impedance mismatch. Variables and database nodes use the same syntax, the same data types, the same operations.

All of this ran on a machine with 8 kilobytes of memory, serving multiple simultaneous users.

1.2. The 1976 Standard

By the early 1970s, multiple vendors had created their own incompatible MUMPS dialects. The MUMPS Development Committee (MDC), formed in 1972, set out to create a common standard. The result was published in January 1976 by the U.S. National Bureau of Standards as **NBS Handbook 118: "MUMPS Language Standard"**. A companion document, the **Programmers' Reference Manual** (MDC/42), was written by Melvin Conway (of "Conway's Law" fame) later that year. Together they form the first complete, formal specification of MUMPS.

The 1976 standard is astonishingly minimal:

- **19 commands:** BREAK, CLOSE, DO, ELSE, FOR, GOTO, HALT, HANG, IF, KILL, LOCK, OPEN, QUIT, READ, SET, USE, VIEW, WRITE, XECUTE
- **12 functions:** \$ASCII, \$CHAR, \$DATA, \$EXTRACT, \$FIND, \$JUSTIFY, \$LENGTH, \$NEXT, \$PIECE, \$RANDOM, \$SELECT, \$TEXT
- **7 special variables:** \$HOROLOGY, \$IO, \$JOB, \$STORAGE, \$TEST, \$X, \$Y

That is the entire language. No JOB command (no programmatic multi-tasking). No NEW command (no local variable scoping — that came in 1984). Subscripts restricted to nonnegative integers for portability. Maximum string length of 255 characters. Yet this was sufficient to build hospital information systems serving millions of patients.

This primer covers exactly this 1976 standard — the essential, crystallized MUMPS, before decades

of extensions.

1.3. Who This Book Is For

This book is for programmers who are fluent in at least one other language and want to understand what MUMPS was, how it worked, and what made it historically significant. You do not need any prior MUMPS experience.

The book serves three purposes:

1. **Introduction:** What MUMPS is and why it matters.
2. **Tutorial:** A progressive, example-driven walkthrough of the language that you can follow along interactively.
3. **Quick reference:** A summary of all commands, functions, and special variables for the 1976 standard.

We focus on single-user operation. Multi-user features like LOCK and device sharing are mentioned where relevant, but their full treatment is deferred.

1.4. Getting Started

To follow along with the examples, start the MUMPS interpreter in interactive mode. You will see a prompt:

```
>
```

At this prompt you can type any MUMPS command and it will be executed immediately. Try:

```
>WRITE "Hello, World!",!  
Hello, World!
```

The **!** after the comma is a format code that produces a new line. We will explain the syntax in detail shortly.

To exit the interpreter, type:

```
>HALT
```

You can also run MUMPS programs in batch mode by loading routine files. But for learning, interactive mode is ideal — it gives you immediate feedback on every command.

2. The Language at a Glance

Before diving into the details, here is a bird's-eye view of MUMPS for the programmer coming from

another language. MUMPS has several features that will feel unfamiliar — deliberately so, because they reflect a different set of design trade-offs.

2.1. Everything Is a String

MUMPS has exactly one data type: the character string. Numbers are simply strings that happen to look numeric. The string `"42"` and the number `42` are the same value. When you perform arithmetic, MUMPS automatically interprets strings as numbers; when you concatenate, it treats values as strings. There are no type declarations, no type errors, and no conversions to worry about.

2.2. No Operator Precedence

All binary operators are evaluated strictly left to right. There is no multiplication-before-addition rule. The expression `2+3*4` evaluates to `20`, not `14`, because `2+3` is computed first (giving `5`), then `5*4` gives `20`. Use parentheses to override this: `2+(3*4)` gives `14`.

This is unfamiliar, but it eliminates an entire class of bugs related to precedence misunderstandings.

2.3. Line-Oriented Structure

A MUMPS program is a sequence of *lines*. Each line has an optional *label* (for reference by `DO`, `GOTO`, or `$TEXT`), followed by a mandatory space separator, followed by zero or more *commands* separated by single spaces:

```
LABEL command1 argument command2 argument ;comment
```

Spaces are syntactically significant. A single space separates the command word from its argument. Two spaces (or end of line) separate an argumentless command from the next command or comment. A semicolon begins a comment that extends to end of line.

2.4. Commands Are Terse

Every command has a full name and a one-letter abbreviation. Both are always valid. `SET X=1` and `S X=1` are identical. `WRITE` and `W`, `IF` and `I`, `FOR` and `F`, and so on. Experienced MUMPS programmers universally use the abbreviations, making MUMPS code extremely dense. In this tutorial we generally use the full command names for clarity, with the abbreviation noted.

2.5. The Persistent Database Is Built In

Any variable whose name starts with a caret (^) is a *global variable*: it is stored persistently on disk, survives process termination, and is shared among all users. Variables without a caret are *local variables*: they exist only in the current process's memory and vanish when the process ends. Global and local variables use exactly the same syntax for subscripts, assignment, and deletion.

3. Variables and Data Types

3.1. Local Variables

A local variable name starts with a letter or %, followed by letters and digits:

```
>SET NAME="Alice"  
>SET AGE=30  
>SET %TEMP=98.6  
>WRITE NAME," is ",AGE," years old.",!  
Alice is 30 years old.
```

Variable names are case-sensitive: `Name`, `NAME`, and `name` are three different variables.

NOTE

In the 1976 standard, portable routine names and labels are limited to 8 significant characters (uppercase only for routine names), but local variable names follow the same `name` syntax: a letter or % followed by alphanumerics.

3.2. Subscripted Variables

Any variable can have subscripts, forming a hierarchical tree:

```
>SET PATIENT("NAME")="Smith, John"  
>SET PATIENT("AGE")=45  
>SET PATIENT("LAB",1)="Glucose: 95"  
>SET PATIENT("LAB",2)="Cholesterol: 210"
```

An unsubscripted variable and its subscripted forms coexist independently. `PATIENT` can hold a value while `PATIENT("NAME")` holds a different one — they are separate nodes in the same tree.

3.3. Strings, Numbers, and Automatic Conversion

Every value in MUMPS is a string. When an arithmetic operator is applied, MUMPS takes the *numeric interpretation* of the string: it scans from left to right, consuming as much of the string as looks like a valid number, and uses that. The rest is ignored.

```
>WRITE +"123ABC"  
123  
>WRITE +"ABC"  
0  
>WRITE +"3.14 is pi"  
3.14  
>WRITE +""  
0
```

The unary `+` forces numeric interpretation. This is how MUMPS handles what other languages call "type conversion" — it simply happens automatically whenever a numeric context requires it.

Numeric values have a canonical form: no leading zeros (except for the number `0` itself), no trailing zeros after the decimal point, and no unnecessary decimal point. The string `"007.10"` has the numeric interpretation `7.1`.

3.4. Truth Values

MUMPS uses `0` for false and `1` for true. The *truth-value interpretation* of any string is: take its numeric interpretation; if the result is `0`, the truth value is `0`; otherwise it is `1`.

```
>WRITE '0
1
>WRITE '1
0
>WRITE '"ABC"
1
```

The apostrophe (`'`) is the NOT operator. `"ABC"` has numeric interpretation `0`, so `'"ABC"` gives `1` — which may be surprising but is perfectly consistent.

4. Expressions and Operators

4.1. Strict Left-to-Right Evaluation

This is the single most important rule to internalize: **all binary operators are at the same precedence level and are evaluated left to right**. Parentheses are the only way to change the order of evaluation.

```
>WRITE 2+3*4
20
>WRITE 2+(3*4)
14
>WRITE 10-2-3
5
>WRITE 1+2_"ABC"
3ABC
```

In the last example, `1+2` gives `3`, then `3_"ABC"` concatenates to produce `"3ABC"`. The underscore `_` is the string concatenation operator.

4.2. Arithmetic Operators

Operator	Meaning	Example	Result
+	Addition	3+4	7
-	Subtraction	10-3	7
*	Multiplication	6*7	42
/	Division	7/2	3.5
\	Integer division	7\2	3
#	Modulo	7#3	1

Integer division (\) truncates toward negative infinity (it computes the floor). Modulo is defined as $A\#B = A - (B * \text{floor}(A/B))$.

4.3. Unary Operators

Operator	Meaning	Example	Result
+	Numeric interpretation	+"42ABC"	42
-	Negation	-3	-3
'	Logical NOT	'0	1

Unary operators bind tighter than binary operators and are applied right to left: '-3 first negates (-3), then takes the truth-value interpretation of -3 (which is nonzero, so 1), then inverts (0).

4.4. Relational Operators

All relational operators return 0 or 1.

Operator	Meaning	Example	Result
=	String equals	"AB"="AB"	1
<	Numeric less than	3<10	1
>	Numeric greater than	10>3	1
]	String follows (ASCII order)	"B"]"A"	1
[	String contains	"ABCDE"["CD"	1
?	Pattern match	"ABC"?3A	1

Each of these can be negated with a preceding apostrophe: '= means "not equal", '<' means "not less than" (i.e., greater than or equal), '>' means "not greater than" (i.e., less than or equal), ']' means "not follows", '[' means "not contains", and '?' means "does not match pattern".

The *follows* operator (]) uses ASCII collating order, which is a character-by-character comparison based on ASCII codes. It is **not** numeric comparison — "9"]"10" is true because "9" follows "1" in ASCII. For numeric comparison, use < and >.

The *contains* operator (`[]`) tests whether the right operand is a substring of the left operand. The empty string is contained in every string.

4.5. Logical Operators

Operator	Meaning	Example	Result
<code>&</code>	AND	<code>1&1</code>	<code>1</code>
<code>!</code>	OR	<code>0!1</code>	<code>1</code>

Negated forms: `'&` is NAND, `'!` is NOR.

Remember: no short-circuit evaluation. Both sides are always evaluated. Since all operators share the same precedence, `A=1!B=2` is parsed as `((A=1)!B)=2`, which is probably not what you mean. Use parentheses: `(A=1)!(B=2)`.

4.6. The Concatenation Operator

The underscore (`_`) concatenates two strings:

```
>SET FIRST="John"
>SET LAST="Smith"
>WRITE FIRST_" "_LAST
John Smith
```

4.7. Pattern Match

The `?` operator tests whether a string matches a pattern. Patterns are built from *pattern atoms*, each consisting of a count (or `.` for "any number") followed by a code or a string literal:

Code	Meaning	Characters
<code>A</code>	Alphabetic	<code>A-Z, a-z</code>
<code>N</code>	Numeric	<code>0-9</code>
<code>U</code>	Uppercase	<code>A-Z</code>
<code>L</code>	Lowercase	<code>a-z</code>
<code>P</code>	Punctuation	includes space
<code>C</code>	Control characters	includes DEL
<code>E</code>	Everything	all 128 ASCII characters

Codes can be combined. `1AN` means "one character that is alphabetic or numeric".

Examples:

```
>WRITE "ABC"?3A
1
>WRITE "AB3"?3A
0
>WRITE "AB3"?2A1N
1
>WRITE "Hello"?1U.AL
1
>WRITE "12345"??.N
1
>WRITE "TEST-1"?1.A1"- "1.N
1
```

The last example reads: "one or more alphabets, then a literal hyphen, then one or more numerics."

5. Commands

5.1. SET — Assign a Value

SET (abbreviation: **S**) assigns a value to one or more variables.

```
>SET X=42
>SET NAME="Alice",AGE=30
>SET (A,B,C)=0
```

The third form is a *multiple SET*: all variables in parentheses receive the same value.

The right-hand side is an expression that is evaluated first; then the result is stored in the named variable(s). Subscript expressions on the left are evaluated before the right-hand expression:

```
>SET I=3
>SET A(I)=I*10
>WRITE A(3)
30
```

5.2. WRITE — Output Data

WRITE (abbreviation: **W**) sends output to the current device.

```
>WRITE "Hello"
Hello
>WRITE 2+2
4
```

```
>WRITE !,"Name: ",NAME,!
Name: Alice
```

WRITE accepts several argument forms, separated by commas:

- **An expression:** outputs its string value.
- **A format code:** **!** outputs a new line (carriage return + line feed). **#** outputs a form feed (clears the screen or starts a new page). **?n** tabs to column *n* (column numbering starts at 0).
- ***n:** outputs the character whose ASCII code is *n*.

These can be freely mixed:

```
>WRITE #,!,"Report",!,"=====",!!
>WRITE ?10,"Column 10",?30,"Column 30",!
>WRITE *7
```

The last example outputs a BEL character (ASCII 7), which may ring the terminal bell.

5.3. READ — Input Data

READ (abbreviation: **R**) reads input from the current device.

```
>READ "Enter your name: ",NAME
Enter your name: Alice
>WRITE "Hello, ",NAME,!
Hello, Alice
```

READ accepts the same format codes and string literal arguments as WRITE (they are output to the terminal as prompts), plus variable arguments that receive input. The input terminates when the user presses Enter (the exact termination procedure is implementation-defined).

READ also supports a timeout:

```
>READ "Answer? ",X:10
```

This waits up to 10 seconds for input. If the user responds in time, the special variable **\$TEST** is set to **1**; if the timeout expires, **\$TEST** is set to **0** and **X** contains whatever partial input was received (possibly the empty string).

A special form reads a single character and returns its ASCII code:

```
>READ *X
```

After the user types **A**, **X** will have the value **65**.

5.4. IF and ELSE — Conditional Execution

IF (abbreviation: **I**) conditionally executes the remainder of the current line.

```
>SET X=5
>IF X>3 WRITE "X is greater than 3",!
X is greater than 3
```

If the condition is false, **everything to the right on that line** is skipped. There is no block structure like braces or **endif** — the scope of IF is always to the end of the current line.

ELSE (abbreviation: **E**) executes the remainder of the line if the most recent IF (or a timed command) set **\$TEST** to 0:

```
>SET X=2
>IF X>3 WRITE "big",!
>ELSE WRITE "small",!
small
```

NOTE

ELSE has no argument—note the **two** spaces between **ELSE** and **WRITE**. An argumentless command requires two spaces before the next command (or end of line).

IF can take multiple arguments separated by commas. They act like a chain of ANDs—if any argument is false, the rest of the line is skipped:

```
>IF X>0,X<100 WRITE "X is between 1 and 99",!
```

This is equivalent to **IF X>0 IF X<100 WRITE ...** or **IF (X>0)&(X<100) WRITE ...**. However, the comma form has a subtle advantage: if the first condition is false, the second is never evaluated, avoiding potential side effects.

5.5. Post-Conditionals

Any command except ELSE, FOR, and IF can be made conditional by appending a colon and a condition directly after the command word:

```
>SET X=5
>WRITE:X>3 "X is big",!
X is big
>SET:X=5 X=0
>WRITE X
0
```

Post-conditionals do not affect **\$TEST**. This is the key difference from IF.

Arguments of DO, GOTO, and XECUTE can also be individually post-conditionalized:

```
>DO SUB1:X=1, SUB2:X=2, SUB3
```

This calls SUB1 only if **X=1**, SUB2 only if **X=2**, and SUB3 unconditionally.

5.6. FOR — Repetition

FOR (abbreviation: **F**) controls repeated execution of the remainder of the current line (its *scope*). Like IF, the scope of FOR extends to end of line.

FOR takes a *loop variable* and one or more *for parameters* separated by commas:

5.6.1. Single value

```
>FOR I=5 WRITE I, " "  
5
```

Executes the scope once with **I** set to **5**.

5.6.2. Start, step (infinite loop)

```
>FOR I=1:1 WRITE I, " " QUIT:I>5  
1 2 3 4 5 6
```

FOR I=1:1 means "start at 1, increment by 1, forever." You must exit explicitly with QUIT or GOTO.

5.6.3. Start, step, stop

```
>FOR I=1:1:5 WRITE I, " "  
1 2 3 4 5
```

FOR I=1:1:5 means "start at 1, step by 1, stop when I exceeds 5." The step can be negative:

```
>FOR I=10:-2:0 WRITE I, " "  
10 8 6 4 2 0
```

5.6.4. Multiple parameters

FOR parameters can be mixed:

```
>FOR I=1:1:3,10,20:5:30 WRITE I," "  
1 2 3 10 20 25 30
```

The parameters are processed left to right, each controlling successive executions of the scope.

5.6.5. Nested FORs

Multiple FORs on the same line are nested, with the rightmost being the innermost:

```
>FOR I=1:1:2 FOR J=1:1:3 WRITE I,"@",J," "  
1@1 1@2 1@3 2@1 2@2 2@3
```

5.6.6. Exiting a FOR

QUIT in the scope of FOR terminates the innermost FOR:

```
>FOR I=1:1:100 QUIT:I>3 WRITE I," "  
1 2 3
```

Note that **QUIT:I>3** must come **before** **WRITE** — the order on the line matters. If **WRITE** came first, the numbers 1 through 100 would be printed with the loop only terminating after the **WRITE** for each iteration.

GOTO in the scope of FOR terminates **all** FORs on that line and transfers control to the specified label.

5.7. DO — Call a Subroutine

DO (abbreviation: **D**) calls a subroutine. In MUMPS, a "subroutine" is simply a label within a routine. Execution continues at the labeled line until a **QUIT** (not in the scope of **FOR**) or end-of-routine is reached, at which point control returns to the command after the **DO**.

Consider this routine:

```
MAIN SET X=10  
DO DOUBLE  
WRITE "Result: ",X,  
HALT  
DOUBLE SET X=X*2  
QUIT
```

Executing **DO MAIN** (or running the routine starting at **MAIN**) would print **Result: 20**.

DO can call into other routines using the caret notation:

```
>DO ^MYROUTINE  
>DO LABEL^OTHERROUTINE  
>DO LABEL+2^ROUTINE
```

The last form means "start at the 2nd line after LABEL in ROUTINE."

Arguments of DO may be listed, with each called in turn:

```
>DO SUB1,SUB2,^EXTERNAL
```

And each argument can be post-conditionalized:

```
>DO PRINTA:X=1,PRINTB:X=2,PRINTC
```

5.8. GOTO — Transfer Control

GOTO (abbreviation: **G**) transfers control to another line. Unlike DO, GOTO does not return—it replaces the current execution point.

```
>GOTO LABEL  
>GOTO LABEL^ROUTINE  
>GOTO LABEL+3^ROUTINE
```

When GOTO is executed inside the scope of FOR, all FORs on that line are terminated.

Like DO, GOTO arguments can be listed and post-conditionalized:

```
>GOTO A:X=1,B:X=2,C
```

At most one argument actually transfers control: the first whose condition (if any) is true.

5.9. QUIT — Return from Subroutine or Exit FOR

QUIT (abbreviation: **Q**) has two behaviors depending on context:

- **Not in FOR scope:** Returns from the current subroutine (DO or XECUTE). If the routine stack becomes empty, the process terminates (equivalent to HALT).
- **In FOR scope:** Terminates the innermost FOR loop.

QUIT takes no arguments, but it can be post-conditionalized:

```
>QUIT:X>100
```

5.10. HALT and HANG

HALT (abbreviation: **H**, argumentless) terminates the process.

HANG (abbreviation: **H**, with argument) suspends execution for a specified number of seconds:

```
>HANG 5
```

The two commands share the abbreviation **H**—the presence or absence of an argument distinguishes them: **H** alone is **HALT**; **H 3** is **HANG 3**.

5.11. XECUTE — Execute a String as Code

XECUTE (abbreviation: **X**) evaluates an expression and executes its string value as a one-line MUMPS subroutine:

```
>SET CMD="WRITE ""Hello from XECUTE"",!"  
>XECUTE CMD  
Hello from XECUTE
```

Note the doubled quotes inside the string literal — each `""` represents a single `"` in the string's value.

XECUTE creates a temporary subroutine scope, so **QUIT** inside the executed string returns to the command after **XECUTE**, not to the line containing it:

```
>XECUTE "SET X=1 QUIT:X=1 WRITE ""not reached"""  
>WRITE X  
1
```

XECUTE is MUMPS's answer to `eval()` in other languages. It is powerful but should be used judiciously.

5.12. KILL — Delete Variables

KILL (abbreviation: **K**) removes variables and their subtrees from storage. It comes in three forms:

5.12.1. Argumentless KILL (Kill All)

```
>KILL
```

Deletes all local variables.

5.12.2. Selective KILL

```
>KILL X  
>KILL ^PATIENT(12345)  
>KILL X,Y,^A(3,4)
```

Deletes the named variable(s) and all descendants.

5.12.3. Exclusive KILL

```
>KILL (A,B)
```

Deletes all local variables **except** A and B (and their descendants). The variables in parentheses are preserved; everything else is removed.

5.13. BREAK

BREAK (abbreviation: **B**, argumentless) suspends execution until a signal from the operating environment (typically a keypress). It is primarily a debugging tool.

```
>BREAK
```

6. String Functions

MUMPS provides a set of built-in functions, each prefixed with **\$** and accepting a parenthesized argument list. Function names can be abbreviated to the **\$** plus the first letter: **\$EXTRACT** and **\$E** are the same function.

6.1. \$LENGTH — String Length

\$LENGTH(expr) returns the number of characters in the value of *expr*. The length of the empty string is 0.

```
>WRITE $LENGTH("Hello")  
5  
>WRITE $L("")  
0
```

6.2. \$EXTRACT — Substring by Position

\$EXTRACT(string,from) returns the single character at position *from* (1-based).
\$EXTRACT(string,from,to) returns the substring from position *from* to *to* inclusive.

```
>SET X="ABCDE"
>WRITE $EXTRACT(X,1)
A
>WRITE $E(X,2,4)
BCD
>WRITE $E(X,3,99)
CDE
```

If the position is out of range, **\$EXTRACT** returns the empty string. There is no "index out of bounds" error.

6.3. \$PIECE — Substring by Delimiter

\$PIECE(string,delimiter,n) returns the *n*-th piece of *string* when split by *delimiter*. An optional fourth argument returns a range of pieces.

```
>SET REC="Smith^John^45^M"
>WRITE $PIECE(REC,"^",1)
Smith
>WRITE $P(REC,"^",2)
John
>WRITE $P(REC,"^",3)
45
>WRITE $P(REC,"^",2,3)
John^45
```

\$PIECE is one of the most heavily used functions in MUMPS programming. Delimited strings are the standard way to store structured data within a single value—essentially a lightweight record format.

6.4. \$FIND — Search for Substring

\$FIND(string,target) searches for *target* in *string*. If found, it returns the position of the character **after** the last character of the match. If not found, it returns **0**. An optional third argument specifies the starting position.

```
>SET X="ABCABC"
>WRITE $FIND(X,"BC")
4
>WRITE $F(X,"BC",4)
7
>WRITE $F(X,"XY")
0
```

The return value is the position **after** the match, which is convenient for chaining: you can pass the

result directly as the starting position for the next search.

6.5. \$JUSTIFY — Right-Justify and Format Numbers

\$JUSTIFY(*expr*,*width*) right-justifies the value of *expr* in a field of *width* characters, padding with spaces on the left.

\$JUSTIFY(*number*,*width*,*decimals*) additionally rounds the number to *decimals* decimal places.

```
>WRITE $JUSTIFY("Hi",10)
      Hi
>WRITE $JUSTIFY(3.14159,10,2)
      3.14
>WRITE $J(100,6,2)
    100.00
```

This is the primary tool for formatted numeric output in MUMPS.

6.6. \$ASCII and \$CHAR — Character Code Conversion

\$ASCII(*string*) returns the ASCII code of the first character of *string* (or -1 if the string is empty). An optional second argument selects a different position.

\$CHAR(*code1*,*code2*,...) returns a string built from the given ASCII codes. Negative codes contribute no character.

```
>WRITE $ASCII("A")
65
>WRITE $A("Hello",2)
101
>WRITE $CHAR(72,101,108,108,111)
Hello
>WRITE $C(65,66,67)
ABC
```

6.7. \$RANDOM — Random Number

\$RANDOM(*n*) returns a pseudo-random integer in the range 0 through *n*-1:

```
>WRITE $RANDOM(10)
7
>WRITE $R(100)
42
```

The argument must be a positive integer.

6.8. \$SELECT — Conditional Expression

`$SELECT(cond1:val1,cond2:val2,...,1:default)` evaluates conditions left to right and returns the value paired with the first true condition. It is an error if no condition is true, so the conventional idiom is to end with `1:default` (since `1` is always true).

```
>SET X=2
>WRITE $SELECT(X=1:"one",X=2:"two",X=3:"three",1:"other")
two
```

`$SELECT` is the MUMPS equivalent of a conditional (ternary) expression.

6.9. \$TEXT — Retrieve Source Line

`$TEXT(label)` returns the source text of the line with the given label in the current routine. `$TEXT(label+n)` returns the n -th line after that label. `$TEXT(+n)` returns the n -th line of the routine.

```
>WRITE $TEXT(MAIN)
MAIN SET X=10
```

If the line does not exist, `$TEXT` returns the empty string.

7. The Global Database

This is the feature that makes MUMPS unique: the hierarchical, persistent, shared database that is built into the language itself.

7.1. Global Variables

Any variable whose name begins with a caret (^) is a *global variable*. Globals are:

- **Persistent:** They survive process termination. When you SET a global, it is written to disk.
- **Shared:** All processes can see and modify the same globals (in a multi-user system).
- **Hierarchical:** Like local variables, globals can have any number of subscripts, forming a tree structure.

```
>SET ^COLOR(1)="Red"
>SET ^COLOR(2)="Green"
>SET ^COLOR(3)="Blue"
>HALT
```

After restarting the interpreter:

```
>WRITE ^COLOR(2)
Green
```

The data is still there. No save command, no file I/O, no serialization. The variable **is** the database.

7.2. Building a Data Model

Here is a small patient registry, built purely with globals:

```
>SET ^PAT(1,"NAME")="Smith, John"
>SET ^PAT(1,"DOB")="1945-03-15"
>SET ^PAT(1,"LAB",1,"DATE")="1976-01-10"
>SET ^PAT(1,"LAB",1,"TEST")="Glucose"
>SET ^PAT(1,"LAB",1,"RESULT")=95
>SET ^PAT(2,"NAME")="Doe, Jane"
>SET ^PAT(2,"DOB")="1960-07-22"
```

This creates a tree like:

```
^PAT
  +-- 1
    |  +-- "NAME" = "Smith, John"
    |  +-- "DOB" = "1945-03-15"
    |  +-- "LAB"
    |      +-- 1
    |          +-- "DATE" = "1976-01-10"
    |          +-- "TEST" = "Glucose"
    |          +-- "RESULT" = 95
  +-- 2
    +-- "NAME" = "Doe, Jane"
    +-- "DOB" = "1960-07-22"
```

No schema was defined. No table was created. The structure emerged simply from the patterns of SET commands.

7.3. \$DATA — Testing Whether a Node Exists

\$DATA(variable) returns an integer describing the state of a node:

Value	Meaning
0	The node does not exist
1	The node exists and has a value, but no descendants

Value	Meaning
10	The node exists, has descendants, but no value of its own
11	The node exists, has both a value and descendants

```
>WRITE $DATA(^PAT)
10
>WRITE $D(^PAT(1,"NAME"))
1
>WRITE $D(^PAT(1,"LAB"))
10
>WRITE $D(^PAT(99))
0
```

A common idiom is to test whether a variable is defined before using it:

```
>IF $DATA(^PAT(1)) WRITE "Patient 1 exists",!
Patient 1 exists
```

7.4. \$NEXT — Walking the Tree

\$NEXT(variable(subscript)) returns the next subscript value at the same level, in ascending numeric order. If there is no next value, it returns **-1**.

NOTE

In the 1976 standard, subscripts are restricted to nonnegative integers for portability. **\$NEXT** works with integer subscripts. Later standards added **\$ORDER** for string-subscript traversal.

To iterate over all patients:

```
>SET ID=0
>FOR SET ID=$NEXT(^PAT(ID)) QUIT:ID=-1 WRITE ID,": ",^PAT(ID,"NAME"),!
1: Smith, John
2: Doe, Jane
```

Note the argumentless FOR — **FOR** followed by two spaces — which creates an infinite loop. The loop body uses **\$NEXT** to advance to the next subscript and QUITs when **-1** is returned (no more entries).

This pattern — an argumentless FOR with **\$NEXT** and a QUIT on **-1** — is the fundamental database traversal idiom in 1976-era MUMPS.

7.5. The Naked Reference

After accessing a subscripted global, MUMPS remembers the "path" to the parent node in a hidden register called the *Naked Indicator*. A subsequent reference of the form `^(subscript)` is interpreted relative to this remembered path.

```
>SET X=^PAT(1,"NAME")
```

After this, the Naked Indicator holds `^PAT(1)`. A naked reference like `^("DOB")` would be equivalent to `^PAT(1,"DOB")`.

The naked reference is a compact notation that was important when every character counted (lines were limited to 255 characters). In practice, it makes code harder to read and is a common source of subtle bugs. We mention it here for completeness but recommend using explicit variable references in new code.

7.6. KILL on Globals

KILL works on globals exactly as it does on locals:

```
>KILL ^PAT(1,"LAB")
```

This removes the "LAB" node and its entire subtree (the lab records for patient 1). The parent node `^PAT(1)` is not affected.

```
>KILL ^PAT
```

This removes the entire patient database.

8. Input/Output

8.1. Formatting Output

The format codes in WRITE (and READ) control the position of output on the current device:

Code	Effect
!	New line (carriage return + line feed). Sets <code>\$X</code> to 0, adds 1 to <code>\$Y</code> .
#	Form feed (new page / clear screen). Sets both <code>\$X</code> and <code>\$Y</code> to 0.

Code	Effect
<code>?n</code>	Tab to column <i>n</i> . If the cursor is already at or past column <i>n</i> , no effect. Otherwise, spaces are output to reach column <i>n</i> . Column numbering starts at 0.

Example — a formatted report:

```
>WRITE #,!,"Patient Report",!
>WRITE "=====",!!
>SET ID=0
>FOR SET ID=$NEXT(^PAT(ID)) QUIT:ID=-1 DO SHOW
>HALT
SHOW WRITE "ID: ",ID,?15,"Name: ",^PAT(ID,"NAME"),!
QUIT
```

8.2. The Special Variables \$X and \$Y

`$X` tracks the current horizontal cursor position (column number) on the current device. `$Y` tracks the current vertical position (line number). Both are updated automatically as characters are output:

```
>WRITE "123" SET Y=$X
```

After this, `Y` is 3 (three characters were output).

A common idiom for line wrapping:

```
>IF $X>72 WRITE !
```

8.3. Device I/O (Overview)

The commands `OPEN`, `USE`, and `CLOSE` manage I/O devices:

- `OPEN device` obtains ownership of a device (e.g., a printer, a file).
- `USE device` makes a device the current target for READ and WRITE.
- `CLOSE device` releases ownership.

The special variable `$IO` contains the identifier of the current device.

In a single-user system, your terminal is typically the default (and only) device. Device management becomes important in multi-user or multi-device configurations, which we defer to a later discussion.

9. Indirection

Indirection is MUMPS's mechanism for dynamic code construction. The `@` operator evaluates an expression and substitutes its value into the surrounding syntax.

9.1. Name Indirection

```
>SET VARNAME="X"  
>SET @VARNAME=42  
>WRITE X  
42
```

`@VARNAME` evaluates `VARNAME` (getting `"X"`) and then uses `"X"` as the variable name. This is equivalent to `SET X=42`.

9.2. Subscript Indirection

```
>SET REF="^PAT(1,""NAME"")"  
>WRITE @REF  
Smith, John
```

9.3. Argument Indirection

For commands that support it, indirection can replace entire arguments or argument lists:

```
>SET ARGS="X=1,Y=2,Z=3"  
>SET @ARGS  
>WRITE X," ",Y," ",Z  
1 2 3
```

9.4. Pattern Indirection

Even patterns can be indirect:

```
>SET PAT="3N"  
>WRITE "123"?@PAT  
1
```

Indirection is powerful but makes code harder to read and debug. Use it when you genuinely need dynamic behavior — for example, when variable names or commands are determined at run time.

10. Special Variables

MUMPS provides seven built-in special variables, each prefixed with **\$**. They are read-only (you cannot SET them).

10.1. \$HOROLOG — Date and Time

\$HOROLOG (abbreviated **\$H**) returns a string of the form **days,seconds** where *days* counts days since December 31, 1840 (day 0,0 is the first second of that day) and *seconds* counts seconds since midnight (0 to 86399).

```
>WRITE $HOROLOG  
67708,51301
```

To extract useful information, use **\$PIECE**:

```
>SET H=$H  
>SET DAYS=$PIECE(H,"",1)  
>SET SECS=$PIECE(H,"",2)  
>SET HOURS=SECS\3600  
>SET MINS=SECS\60#60  
>WRITE "Seconds since midnight: ",SECS,!  
>WRITE "Time: ",HOURS,":",MINS,!
```

The peculiar epoch (1840-12-31) was chosen because it precedes any date likely to appear in medical records, and the date arithmetic works out conveniently for leap year calculations.

10.2. \$IO — Current Device

\$IO (abbreviated **\$I**) returns the identifier of the current I/O device, as set by USE or the system default.

10.3. \$JOB — Process Identifier

\$JOB (abbreviated **\$J**) returns a positive integer that uniquely identifies the current process. It remains constant throughout the life of the process.

A common use is to create process-specific temporary globals:

```
>SET ^TEMP($JOB,"SCRATCH")="work in progress"
```

10.4. \$STORAGE — Available Space

\$STORAGE (abbreviated **\$S**) returns an integer representing the number of characters of free space

available. Its exact meaning is implementation-defined, but it measures how much room remains for local variables and routine code.

```
>IF $STORAGE<100 GOTO ^OVERFLOW
```

10.5. \$TEST — Last Test Result

\$TEST (abbreviated **\$T**) contains the truth value (0 or 1) set by the most recent IF with an argument, or by an OPEN, LOCK, or READ with a timeout.

```
>READ "Input: ",X:5  
>IF $TEST WRITE "Got input: ",X,!  
>ELSE WRITE "Timed out.",!
```

10.6. \$X and \$Y — Cursor Position

\$X and **\$Y** return the current horizontal and vertical cursor positions on the current device, as described in the I/O section.

11. Routines and Program Structure

11.1. Routine Structure

A MUMPS *routine* is the unit of program storage and interchange. It consists of a sequence of lines, each terminated by an end-of-line marker. The routine as a whole ends with an end-of-routine marker.

In interchange format:

```
ROUTINENAME  
LABEL1 command command ;comment  
    command command  
LABEL2 command  
QUIT
```

The first line (the routine head) contains only the routine name. The remaining lines form the routine body.

11.2. Lines and Labels

Each line has an optional label at the left margin, followed by a space separator (**1s**), followed by commands:

```
CALC SET X=A+B
WRITE "Result: ",X,!
QUIT
```

The line **CALC** has a label. The next two lines have no labels (they begin with a space). Labels are used as targets for DO, GOTO, and \$TEXT.

Labels can be either names (like **CALC**) or integer literals (like **1**, **42**). When a label is an integer, leading zeros are significant: label **01** is different from label **1**.

11.3. Entry References

DO and GOTO use *entry references* to specify where to transfer control:

Form	Meaning
DO LABEL	Call the line labeled LABEL in the current routine
DO ^ROUTINE	Call the first line of ROUTINE
DO LABEL^ROUTINE	Call the line labeled LABEL in ROUTINE
DO LABEL+3^ROUTINE	Call the 3rd line after LABEL in ROUTINE

GOTO uses the same forms, but transfers control rather than calling a subroutine.

11.4. Command Abbreviations

For reference, here is the complete list of command abbreviations:

Command	Abbreviation
BREAK	B
CLOSE	C
DO	D
ELSE	E
FOR	F
GOTO	G
HALT	H (no argument)
HANG	H (with argument)
IF	I
KILL	K
LOCK	L
OPEN	O

Command	Abbreviation
QUIT	Q
READ	R
SET	S
USE	U
VIEW	V
WRITE	W
XECUTE	X

No other abbreviation is valid. **GO** for GOTO, **WR** for WRITE, etc., are all errors.

12. Putting It All Together

Let us build a small but complete application: a phone directory that stores names and phone numbers in global variables and lets the user look them up, add entries, and list all entries.

```
PHONE ; Phone directory application
  WRITE #,!,"=== Phone Directory ===",!
MENU WRITE !,"Options: (A)dd (L)ookup (D)irectory (Q)uit",!
  READ "Choice: ",CH QUIT:CH=""
  IF CH="A"!(CH="a") DO ADD GOTO MENU
  IF CH="L"!(CH="l") DO LOOKUP GOTO MENU
  IF CH="D"!(CH="d") DO LIST GOTO MENU
  IF CH="Q"!(CH="q") QUIT
  WRITE "Invalid choice.",! GOTO MENU
;
ADD WRITE !,"Add new entry:",!
  READ "  Name: ",NAME QUIT:NAME=""
  READ "  Phone: ",PHONE QUIT:PHONE=""
  SET ^PHONE(NAME)=PHONE
  WRITE "  Saved.",!
  QUIT
;
LOOKUP WRITE !,"Look up entry:",!
  READ "  Name: ",NAME QUIT:NAME=""
  IF $DATA(^PHONE(NAME)) WRITE "  Phone: ",^PHONE(NAME),!
  ELSE WRITE "  Not found.",!
  QUIT
;
LIST WRITE !,"All entries:",!
  SET NAME=""
  FOR SET NAME=$NEXT(^PHONE(NAME)) QUIT:NAME=-1 DO
    . WRITE "  ",NAME,?25,^PHONE(NAME),!
  QUIT
```

This small program demonstrates most of the features covered in this tutorial:

- Global variables (^PHONE) for persistent storage
- READ for user input with prompts
- IF/ELSE for branching
- DO for subroutine calls
- GOTO for returning to the menu
- FOR with \$NEXT for iterating over the database
- \$DATA for testing whether an entry exists
- Post-conditionals (QUIT:NAME="") for early exit
- Format codes (!, #, ?25) for output formatting

The data persists across sessions. You can add entries, quit, restart the interpreter, and the entries are still there — because globals are the database.

13. Command Reference

This section provides a concise reference for all 19 commands defined in the 1976 standard. Each entry lists the full syntax, abbreviation, and key behaviors. For tutorial-style explanations and examples, see the earlier chapters.

13.1. BREAK

Abbreviation: B

Syntax: B[BREAK][:tvexpr]

Suspends execution until a signal from the environment (typically a keypress). The argument syntax, if any, is implementation-defined. BREAK can be post-conditionalized. It is primarily used for debugging.

13.2. CLOSE

Abbreviation: C

Syntax: C[CLOSE][:tvexpr] expr[:deviceparams][,...]

Releases ownership of the device identified by *expr*. If *deviceparams* are present, they specify termination procedures in a format defined by the implementation. If the closed device is the current device, the implementation may implicitly OPEN and USE a default device.

```
CLOSE 3
C 3:(parameters)
```

13.3. DO

Abbreviation: **D**

Syntax: **D**[0][:*tvexpr*] *entryref*[:*tvexpr*][, ...]

Calls a subroutine. Execution continues at the line indicated by *entryref* until QUIT (outside FOR scope) or end-of-routine, then returns to the command after DO.

entryref may be: **label**, **label+offset**, **^routine**, **label^routine**, or **label+offset^routine**.

Both the command and individual arguments may be post-conditionalized. Arguments may be listed.

Argumentless DO (followed by two spaces or end of line) is not defined in the 1976 standard.

```
DO CALC
D INIT, ^SETUP, PROC ^LIB
D PRINTA:X=1, PRINTB:X=2
```

13.4. ELSE

Abbreviation: **E**

Syntax: **`E[LSE]** **`** (no argument; note two spaces after the command)

Executes the remainder of the line if **\$TEST** is 0. ELSE takes no arguments and cannot be post-conditionalized.

ELSE is equivalent to **IF '\$TEST** but does not alter **\$TEST**.

```
IF X>3 WRITE "big",!
ELSE WRITE "small",!
```

13.5. FOR

Abbreviation: **F**

Syntax: **F**[OR] **lvn=forparam**[, **forparam**...]

or argumentless: **`F**[OR] **`** (two spaces; infinite loop)

FOR cannot be post-conditionalized. Its scope is the remainder of the current line.

Each *forparam* has one of three forms:

- **expr** — execute scope once with **lvn** set to value of *expr*.
- **expr1:expr2** — start at *expr1*, increment by *expr2* each iteration, loop forever (exit via QUIT or

GOTO).

- **expr1:expr2:expr3**—start at *expr1*, step by *expr2*, terminate when *lvn* exceeds *expr3* (or falls below *expr3* if step is negative).

Multiple *forparams* are processed left to right.

```
FOR I=1:1:10 WRITE I," "  
F I=1:1 QUIT:I>5 WRITE I," "  
FOR I=1,3,5,7:2:15 WRITE I," "  
FOR READ X QUIT:X="" SET ^DATA(X)=""
```

13.6. GOTO

Abbreviation: **G**

Syntax: **G[OTO][:tvexpr] entryref[:tvexpr][,...]**

Transfers control to the line indicated by *entryref*. Unlike DO, GOTO does not push the return stack—control does not return. When executed in the scope of FOR, all FORs on the line are terminated.

Both the command and individual arguments may be post-conditionalized. At most one argument actually executes (the first whose condition is true).

```
GOTO LOOP  
G A:X=1,B:X=2,C  
G INIT^SETUP
```

13.7. HALT

Abbreviation: **H** (argumentless)

Syntax: **H[ALT][:tvexpr]**

Terminates the current process. HALT takes no arguments; the absence of an argument is what distinguishes it from HANG.

```
HALT  
H
```

13.8. HANG

Abbreviation: **H** (with argument)

Syntax: **H[ANG][:tvexpr] numexpr[,...]**

Suspends execution for *numexpr* seconds. If the value is 0 or negative, there is no pause. Arguments may be listed.

```
HANG 5  
H 0.5,1,2
```

13.9. IF

Abbreviation: **I**

Syntax: **I[F] tvexpr[,tvexpr...]**

or argumentless: **`I[F] `** (two spaces)

If *tvexpr* is true, the remainder of the line is executed; if false, the remainder of the line is skipped and **\$TEST** is set to 0.

With multiple arguments (comma-separated), each is evaluated left to right. If any is false, the rest of the line is skipped. Each true argument sets **\$TEST** to 1; the first false argument sets **\$TEST** to 0.

Argumentless IF tests **\$TEST**: if it is 1, the line continues; if 0, the line is skipped. Argumentless IF does not alter **\$TEST**.

IF cannot be post-conditionalized.

```
IF X>3 WRITE "big",!  
I X>0,X<100 WRITE "in range",!  
IF WRITE "previous test was true",!
```

13.10. KILL

Abbreviation: **K**

Syntax: **K[ILL][:tvexpr] glvn[,...]**

or exclusive: **K[ILL][:tvexpr] (lvn[,...])**

or argumentless: **K[ILL][:tvexpr]**

- **Selective:** **KILL X** deletes **X** and all its descendants.
- **Exclusive:** **KILL (A,B)** deletes all local variables except **A** and **B** (and their descendants). Only local variable names are permitted in the exclusive form.
- **Argumentless:** **KILL** deletes all local variables.

KILL on a global (**KILL ^X**) removes the global node and its entire subtree from disk.

```
KILL X,Y,^A(3)  
K (KEEP1,KEEP2)
```

13.11. LOCK

Abbreviation: L

Syntax: L[OCK][:tvexpr] nref[:timeout][,...]

or argumentless: L[OCK][:tvexpr]

LOCK provides inter-process coordination by creating advisory locks on named resources. In a single-user system, LOCK has no practical effect.

Argumentless LOCK releases all locks held by the process.

With arguments, LOCK first releases all currently held locks, then attempts to acquire each named lock in order. If a timeout is present and the lock cannot be acquired within the specified time, **\$TEST** is set to 0.

NOTE LOCK is deferred in this primer. It becomes relevant in multi-user configurations.

13.12. OPEN

Abbreviation: O

Syntax: O[PEN][:tvexpr] expr[:deviceparams][:timeout][,...]

Acquires ownership of the device identified by *expr*. If a timeout is present and the device cannot be opened in time, **\$TEST** is set to 0. *deviceparams* specify device-specific initialization in a format defined by the implementation.

```
OPEN 3
O 3::10
```

13.13. QUIT

Abbreviation: Q

Syntax: Q[UIT][:tvexpr]

QUIT takes no arguments. Its behavior depends on context:

- **Outside FOR scope:** Returns from the current DO or XECUTE subroutine call. If the routine stack is empty, the process terminates.
- **Inside FOR scope:** Terminates the innermost FOR loop. If the line has nested FORs, only the innermost is terminated.

QUIT can be post-conditionalized.

```
QUIT
Q:X>100
```

13.14. READ

Abbreviation: **R**

Syntax: **R[EAD][:tvexpr] readarg[,...]**

Each *readarg* is one of:

- **"string"** — output the string literal (as a prompt).
- **format** — output format codes (**!**, **#**, **?n**).
- **lvn** — read a data string into the local variable. Terminated by an implementation-defined procedure (typically Enter).
- **lvn:timeout** — read with a timeout of *timeout* seconds. Sets **\$TEST** to 1 if terminated normally, 0 if timed out.
- ***lvn** — read a single character and store its ASCII code (integer) in *lvn*. Returns -1 if timed out.
- ***lvn:timeout** — single-character read with timeout.

Arguments may be listed freely, mixing prompts, format codes, and variable reads.

```
READ "Name: ",NAME
R !,"Age: ",AGE,!,"City: ",CITY
R "Answer? ",X:10
R *CH
```

13.15. SET

Abbreviation: **S**

Syntax: **S[ET][:tvexpr] setarg[,...]**

Each *setarg* is one of:

- **glvn=expr** — evaluate *expr* and assign its value to the variable.
- **(glvn[,...])=expr** — evaluate *expr* and assign its value to all listed variables (multiple SET).

Evaluation order within each argument: left-hand subscripts are evaluated first (left to right), then the right-hand expression, then the assignment is made. Each argument is fully completed before the next begins.

SET can be post-conditionalized. Arguments may be listed.

```
SET X=42
S NAME="Alice",AGE=30
S (A,B,C)=0
S ^PAT(ID,"NAME")=NAME
S:X>0 Y=X*2
```

13.16. USE

Abbreviation: **U**

Syntax: **U[SE][:tvexpr] expr[:deviceparams][,...]**

Makes the device identified by *expr* the current device for subsequent READ and WRITE operations. The device must have been previously OPENed (or be the default device). The values of **\$X** and **\$Y** are saved for the old device and restored from the new device's stored values.

```
USE 3
U 0
```

13.17. VIEW

Abbreviation: **V**

Syntax: **V[IEW][:tvexpr] args...** (implementation-defined)

VIEW is reserved for implementation-specific commands. Its argument syntax is defined by the implementor. Every implementation must recognize the VIEW command, even if it does nothing.

Programs designed for portability should not use VIEW.

13.18. WRITE

Abbreviation: **W**

Syntax: **W[RITE][:tvexpr] writearg[,...]**

Each *writearg* is one of:

- **expr** — output the string value of *expr* to the current device.
- **format** — output format codes: **!** (new line), **#** (form feed), **?intexpr** (tab to column *intexpr*).
- ***intexpr** — output the character whose ASCII code is the value of *intexpr*. The exact interpretation is implementation-defined.

WRITE can be post-conditionalized. Arguments may be listed.

```
WRITE "Hello, World!",!
```

```
W !,"Name: ",NAME,?30,"Age: ",AGE,!
W *7
W:X>0 $JUSTIFY(X,10,2),!
```

13.19. XECUTE

Abbreviation: **X**

Syntax: **X**[**ECUTE**][:**tvexpr**] **expr**[:**tvexpr**][,⋯]

Evaluates *expr* and executes its string value as a one-line MUMPS subroutine. The executed code runs in its own subroutine context — QUIT within the executed string returns to the command after XECUTE.

Both the command and individual arguments may be post-conditionalized. Arguments may be listed.

The routine-body context of the executed code (for **\$TEXT** and local entry references) is the routine containing the XECUTE.

```
XECUTE "SET X=1 WRITE X,!"
X "GOTO "_LABEL
X CMD1:X=1,CMD2:X=2,CMD3
```

14. Function Reference

All functions are prefixed with **\$** and can be abbreviated to **\$** plus the first letter of the name. This section provides the complete reference for all 12 functions defined in the 1976 standard.

14.1. \$ASCII

Abbreviation: **\$A**

Syntax: **\$A**[**SCII**](**expr**[:**intexpr**])

Returns the decimal ASCII code of a character in *expr*.

- If *intexpr* is absent, the first character is used.
- If *intexpr* is present, the character at position *intexpr* is used.
- If the selected position is empty (empty string, or position out of range), returns **-1**.

```
$A("ABC")      = 65
$A("ABC",2)    = 66
$A("")         = -1
$A("AB",5)     = -1
```

14.2. \$CHAR

Abbreviation: `$C`

Syntax: `$C[HAR](intexpr[,intexpr...])`

Returns a string built from the given ASCII codes. Each *intexpr* in the range [0,127] contributes its corresponding ASCII character. Negative values contribute no character. Values above 127 are erroneous.

```
$C(65)      = "A"
$C(65,66,67) = "ABC"
$C(65,-1,66) = "AB"
$C(0)       = the ASCII NUL character
```

14.3. \$DATA

Abbreviation: `$D`

Syntax: `$D[ATA](glvn)`

Returns a nonnegative integer characterizing the state of the named variable:

Value	Meaning
0	Node does not exist
1	Node has a value, no descendants
10	Node has descendants, no value of its own
11	Node has both a value and descendants

The argument may be a local or global variable, subscripted or unsubscripted. If the argument contains a global reference, the Naked Indicator is updated.

```
SET X=1      $D(X)=1
SET A(1,2)=3  $D(A)=10  $D(A(1))=10  $D(A(1,2))=1
KILL A       $D(A)=0
```

14.4. \$EXTRACT

Abbreviation: `$E`

Syntax: `$E[XTRACT](expr,intexpr1[,intexpr2])`

Returns a substring of *expr* by character position (1-based).

- Two-argument form: returns the single character at position *intexpr1*. Returns empty string if

out of range.

- Three-argument form: returns characters from position *intexpr1* to *intexpr2* inclusive. If *intexpr1* > *intexpr2*, returns empty string. Positions beyond the string length are silently ignored.

```
$E("ABCDE",1)      = "A"  
$E("ABCDE",2,4)     = "BCD"  
$E("ABCDE",3,99)    = "CDE"  
$E("ABCDE",0,100)   = "ABCDE"  
$E("ABCDE",4,2)     = ""
```

14.5. \$FIND

Abbreviation: **\$F**

Syntax: **\$F[IND](expr1,expr2[,intexpr3])**

Searches for *expr2* as a substring of *expr1*. Returns the position number of the character **immediately after** the rightmost character of the first match found, or 0 if no match. The search starts at position *intexpr3* (default 1).

If *expr2* is the empty string, returns *intexpr3* (default 1).

```
$F("ABCABC","BC")   = 4  
$F("ABCABC","BC",4) = 7  
$F("ABCABC","XY")   = 0  
$F("ABC","")        = 1
```

14.6. \$JUSTIFY

Abbreviation: **\$J**

Syntax: **\$J[JUSTIFY](expr,intexpr)**
or: **\$J[JUSTIFY](numexpr,intexpr1,intexpr2)**

Two-argument form: right-justifies the value of *expr* in a field of *intexpr* characters, padding with spaces on the left. If the value is already wider than *intexpr*, it is returned unchanged.

Three-argument form: rounds *numexpr* to *intexpr2* decimal places (including trailing zeros), then right-justifies in a field of *intexpr1* characters.

```
$J("Hi",10)         = "          Hi"  
$J(3.14159,10,2)    = "          3.14"  
$J(100,8,2)         = "    100.00"  
$J(3.14159,1,2)     = "3.14"
```

14.7. \$LENGTH

Abbreviation: `$L`

Syntax: `$L[LENGTH](expr)`

Returns the number of characters in the value of *expr*. The length of the empty string is 0.

```
$L("Hello") = 5
$L("")      = 0
$L(123)     = 3
```

14.8. \$NEXT

Abbreviation: `$N`

Syntax: `$N[EXT](glvn)`

Returns the next subscript value at the same level, in ascending integer order. The argument must be a subscripted variable. Only the last subscript is varied; all preceding subscripts are held constant.

If the last subscript value is -1, returns the first (lowest) subscript at that level (if any). If no higher subscript exists, returns -1.

In the 1976 standard, subscripts are restricted to nonnegative integers for portable programs, and `$NEXT` operates on integer subscripts.

```
SET ^A(2)=1,^A(5)=1,^A(9)=1
$N(^A(0)) = 2
$N(^A(2)) = 5
$N(^A(5)) = 9
$N(^A(9)) = -1
$N(^A(-1)) = 2
```

14.9. \$PIECE

Abbreviation: `$P`

Syntax: `$P[IECE](expr1,expr2,intexpr1[,intexpr2])`

Splits *expr1* by the delimiter *expr2* and returns pieces by number.

- Three-argument form: returns the *intexpr1*-th piece. Pieces are numbered starting at 1.
- Four-argument form: returns pieces *intexpr1* through *intexpr2*, with the delimiter restored between them.

If the piece number is out of range (no such piece exists), the empty string is returned.

```
$P("A^B^C", "^", 1)    = "A"
$P("A^B^C", "^", 2)    = "B"
$P("A^B^C", "^", 4)    = ""
$P("A^B^C", "^", 1, 2) = "A^B"
$P("A::C", ":", 2)      = ""
$P("ABC", "^", 1)      = "ABC"
```

14.10. \$RANDOM

Abbreviation: `$R`

Syntax: `$R[ANDOM](intexpr)`

Returns a pseudo-random integer uniformly distributed in the range `[0, intexpr-1]`. The argument must be a positive integer (`>= 1`).

```
$R(10)  returns an integer from 0 to 9
$R(1)   always returns 0
$R(100) returns an integer from 0 to 99
```

14.11. \$SELECT

Abbreviation: `$S`

Syntax: `$S[ELECT](tvexpr1:expr1[, tvexpr2:expr2...])`

Evaluates the truth-value expressions left to right. Returns the value of the *expr* paired with the first true *tvexpr*. Only that single *expr* is evaluated.

It is an error if no *tvexpr* is true. The conventional idiom is to end with `1:default`.

```
$S(X=1:"one", X=2:"two", 1:"other")
$S(X>0:X, 1:-X)
```

14.12. \$TEXT

Abbreviation: `$T`

Syntax: `$T[EXT](lineref)` or `$T[EXT](+intexpr)`

Returns the source text of a line in the current routine.

- `$T(label)` — the line with the defining occurrence of *label*.

- `$T(label+n)` — the n -th line after *label*.
- `$T(+n)` — the n -th line of the routine (first line is +1).

The line-start character is replaced by one space. The end-of-line marker is removed. If the specified line does not exist, the empty string is returned.

```
$T(MAIN)      returns "MAIN SET X=10"
$T(MAIN+1)    returns " DO DOUBLE"
$T(+1)        returns the first line of the routine
$T(+999)      returns "" if the routine has fewer than 999 lines
```

15. Special Variables Reference

All special variables are prefixed with `$` and can be abbreviated to `$` plus the first letter. They are read-only: you cannot assign to them with SET.

15.1. \$HOROLOG

Abbreviation: `$H`

Returns a string of the form `D,S` where D is an integer counting days since December 31, 1840 and S is an integer (0 to 86399) counting seconds since midnight.

The epoch is defined so that the first second of December 31, 1840 is `0,0`. S resets to 0 at midnight with a carry into D .

15.2. \$IO

Abbreviation: `$I`

Returns the identifier of the current I/O device. This value is the same as the device specifier most recently provided to USE, or the default device if no USE has been executed.

15.3. \$JOB

Abbreviation: `$J`

Returns a positive integer uniquely identifying the current process. The value is constant throughout the life of the process.

15.4. \$STORAGE

Abbreviation: `$S`

Returns an integer representing the number of characters of free space available for local variables and routine storage. The exact computation is implementation-defined.

15.5. \$TEST

Abbreviation: `$T`

Contains `0` or `1`. Set by:

- `IF` with argument(s): set to 1 if the (last) condition is true, 0 if false.
- `READ` with timeout: set to 1 if input was explicitly terminated before timeout, 0 otherwise.
- `OPEN` with timeout: set to 1 if the device was successfully opened, 0 otherwise.
- `LOCK` with timeout: set to 1 if the lock was acquired, 0 otherwise.

`$TEST` is not altered by post-conditionals, argumentless `IF`, or `ELSE`.

15.6. \$X

Returns a nonnegative integer approximating the horizontal cursor position (column number) on the current device. Column 0 is the left margin. Updated by `READ` and `WRITE`: each graphic adds 1; carriage return and form feed reset to 0; backspace decrements (minimum 0).

15.7. \$Y

Returns a nonnegative integer approximating the vertical cursor position (line number) on the current device. Line 0 is the top of the page. Updated by `READ` and `WRITE`: line feed adds 1; form feed resets to 0.

16. Operators Reference

16.1. Unary Operators

Unary operators bind tighter than binary operators and are applied right to left.

Operator	Name	Operand Interpretation	Result
<code>+</code>	Plus	Numeric	Numeric interpretation of operand
<code>-</code>	Minus	Numeric	Negated numeric interpretation
<code>'</code>	Not	Truth-value	1 if operand is 0, else 0

16.2. Binary Operators

All binary operators share the same precedence and are evaluated strictly left to right. Parentheses override this order.

16.2.1. Arithmetic

Operator	Name	Result
+	Add	Numeric sum
-	Subtract	Numeric difference
*	Multiply	Numeric product
/	Divide	Numeric quotient; division by zero is an error
\	Integer divide	Integer interpretation of / result (floor)
#	Modulo	$A\#B = A - (B * \text{floor}(A/B))$; B must be nonzero

16.2.2. String

Operator	Name	Result
-	Concatenate	Left operand followed by right operand

16.2.3. Relational (return 0 or 1)

Operator	Name	Operand Interpretation	True When
=	Equals	String	Operands are identical strings
<	Less than	Numeric	Left < Right
>	Greater than	Numeric	Left > Right
]	Follows	String (ASCII)	Left follows Right in ASCII order
[	Contains	String	Right is a substring of Left
?	Pattern match	Left: string; Right: pattern	Left matches pattern

Each relational operator can be negated with a preceding apostrophe: '=', '<', '>', ']', '[', '?.

16.2.4. Logical (return 0 or 1)

Operator	Name	Operand Interpretation	True When
&	And	Truth-value	Both operands are true (nonzero)
!	Or	Truth-value	At least one operand is true

Negated forms: '⋈' (NAND), '!' (NOR).

17. Syntax Summary

17.1. Line Structure

```
[label] ls command[ command]...[ ;comment] eol
```

- *ls* is a single space.
- Commands are separated by a single space.
- If a command has no arguments, it is followed by two spaces before the next command (or end of line).
- A semicolon begins a comment that extends to end of line.

17.2. Routine Structure

```
routinename eol
line [line]... eor
```

- The first line (routine head) contains only the routine name.
- In program interchange: *eol* is CR LF; *eor* is CR FF.

17.3. Name Syntax

```
name ::= (%|alpha)(alpha|digit)*
```

A name starts with % or a letter, followed by any number of letters and digits. Names are case-sensitive.

17.4. Variable Syntax

- **Local:** *name* or *name(expr[,expr...])*

- **Global:** `^name` or `^name(expr[,expr...])` or `^(expr[,expr...])` (naked reference)

17.5. Numeric Literal Syntax

```
numlit ::= [digits][.digits][E[+|-]digits]
```

Numeric values have a canonical form: no leading zeros (except `0` itself), no trailing zeros after the decimal point, no unnecessary decimal point.

17.6. String Literal Syntax

A string literal is enclosed in double quotes. A literal double quote within the string is represented by two adjacent double quotes.

```
"Hello"
""
"He said ""hi"""
```

17.7. Entry Reference Syntax

```
entryref ::= lineref [^routineref] | ^routineref
lineref  ::= label [+intexpr]
```

Appendix A: ASCII Code Table

The following table lists the printable ASCII characters (codes 32-126) and their decimal values, which are relevant to `$ASCII` and `$CHAR`.

32 SP	48 0	64 @	80 P	96 `	112 p
33 !	49 1	65 A	81 Q	97 a	113 q
34 "	50 2	66 B	82 R	98 b	114 r
35 #	51 3	67 C	83 S	99 c	115 s
36 \$	52 4	68 D	84 T	100 d	116 t
37 %	53 5	69 E	85 U	101 e	117 u
38 &	54 6	70 F	86 V	102 f	118 v
39 '	55 7	71 G	87 W	103 g	119 w
40 (	56 8	72 H	88 X	104 h	120 x
41)	57 9	73 I	89 Y	105 i	121 y
42 *	58 :	74 J	90 Z	106 j	122 z
43 +	59 ;	75 K	91 [	107 k	123 {
44 ,	60 <	76 L	92 \	108 l	124
45 -	61 =	77 M	93]	109 m	125 }
46 .	62 >	78 N	94 ^	110 n	126 ~

Selected control characters (codes 0-31):

Code	Name	Abbrev.	Relevance
0	Null	NUL	<code>\$CHAR(0)</code>
7	Bell	BEL	<code>WRITE *7</code> rings terminal bell
8	Backspace	BS	Decrements <code>\$X</code>
9	Tab	HT	
10	Line Feed	LF	Increments <code>\$Y</code> ; part of <code>!</code> format
12	Form Feed	FF	Resets <code>\$X</code> and <code>\$Y</code> ; part of <code>#</code> format
13	Carriage Return	CR	Resets <code>\$X</code> ; part of <code>!</code> format
27	Escape	ESC	
127	Delete	DEL	Control character

Appendix B: Glossary

Ascending Order

For integer subscripts, the natural numeric order: 0, 1, 2, 3, ... For strings, the ASCII collating sequence.

Caret

The `^` character (ASCII 94). Prefixes global variable names.

Current Device

The I/O device to which `READ` and `WRITE` are directed. Identified by `$IO`. Changed by `USE`.

D Attribute

An integer (0, 1, 10, or 11) associated with each node in the variable tree, returned by `$DATA`. Encodes whether the node has a defined value (units digit) and whether it has descendants (tens digit).

Entry Reference

A specification of a line in a routine, used by `DO`, `GOTO`, and `$TEXT`. May include a label, an offset, and a routine name.

Global Variable

A variable whose name begins with `^`. Stored persistently on disk, shared among all processes, and organized as a hierarchical tree.

Label

A name or integer literal at the beginning of a line that identifies the line for reference by DO, GOTO, or \$TEXT.

Level Indicator (Line Level)

In the 1976 standard, a dot-based notation (not used in NBS 118 but present in some implementations) to indicate block structure.

Local Variable

A variable without the `^` prefix. Exists only in the current process's memory; lost when the process terminates.

Naked Indicator

A hidden register that remembers the global name and all but the last subscript of the most recently executed global reference. Used to interpret naked references of the form `^(subscript)`.

Naked Reference

A global variable reference of the form `^(expr[, ...])` that relies on the Naked Indicator for the global name and leading subscripts.

Node

A point in the variable tree identified by a name and zero or more subscripts. Each node may have a value, descendants, both, or neither.

Partition

In the MUMPS system model, the execution context of a single process, containing local variables, the routine stack, the Test Switch, and the Naked Indicator.

Post-Conditional

A `:tvexpr` appended to a command word or (for DO, GOTO, XECUTE) an argument, which causes conditional execution without affecting `$TEST`.

Routine

The unit of program storage and interchange. Consists of a routine head (the routine name) followed by a routine body (a sequence of lines).

Routine Stack

A stack that records return points for DO and XECUTE calls. QUIT pops the stack.

Scope (of FOR or IF)

The remainder of the current line. FOR and IF control execution of everything to their right on the same line.

Special Variable

A read-only system variable prefixed with `$` (e.g., `$HOROLOG`, `$TEST`). Provides access to system state.

Storage Reference

A syntactic construct denoting a variable node: either a local variable name, a global variable name, or a naked reference, any of which may be subscripted.

String

The sole data type in MUMPS. All values, including numbers, are represented as character strings.

Subscript

An expression enclosed in parentheses after a variable name, used to select a descendant node. Multiple subscripts, separated by commas, identify a path through the tree.

Test Switch

A Boolean register (0 or 1) whose content is available via **\$TEST**. Set by IF (with arguments), and by OPEN, LOCK, and READ when a timeout is used.

Timeout

A **:numexpr** appended to arguments of READ, OPEN, or LOCK, specifying a maximum wait time in seconds. Sets **\$TEST** on completion.

Truth Value

0 (false) or **1** (true). The truth-value interpretation of any string is: take its numeric interpretation; if the result is **0**, the truth value is **0**; otherwise it is **1**.

18. What We Have Not Covered

This primer focuses on the core language for single-user operation. The following topics are deferred:

- **Multi-user operation:** LOCK semantics, partitions, job management, concurrent global access.
- **Device parameters:** Implementation-specific arguments to OPEN, USE, and CLOSE.
- **VIEW and \$VIEW:** Implementation-specific commands and functions for accessing system internals.
- **Z extensions:** Implementation-specific commands (**Z...**) and functions (**\$Z...**).
- **Later standards:** The 1984 ANSI standard added **NEW** (local variable scoping) and **\$ORDER** (string-subscript traversal). The 1990 and 1995 standards added many more features. These are outside the scope of this primer.

References

- [nbs118] Joseph T. O'Neill, ed. *MUMPS Language Standard*. National Bureau of Standards Handbook 118, January 1976.
- [mdc42] Melvin E. Conway. *MUMPS Programmers' Reference Manual*. MUMPS Development Committee, MDC/42, September 1976.

- [wasserman75] Anthony I. Wasserman, David D. Sherertz, Charles P. Rogerson. *MUMPS Globals and their Implementation*. 1975.